Ray Tracing II

SOCS Teaching Assistant Excellence Award

- Had a great experience with one of your TAs?
- Nominate them for a TA Award!

Nomination Information:

https://www.cs.mcgill.ca/academic/ta/taawards

Criteria: organized, knowledgeable, effective, accessible, inspiring, role model...

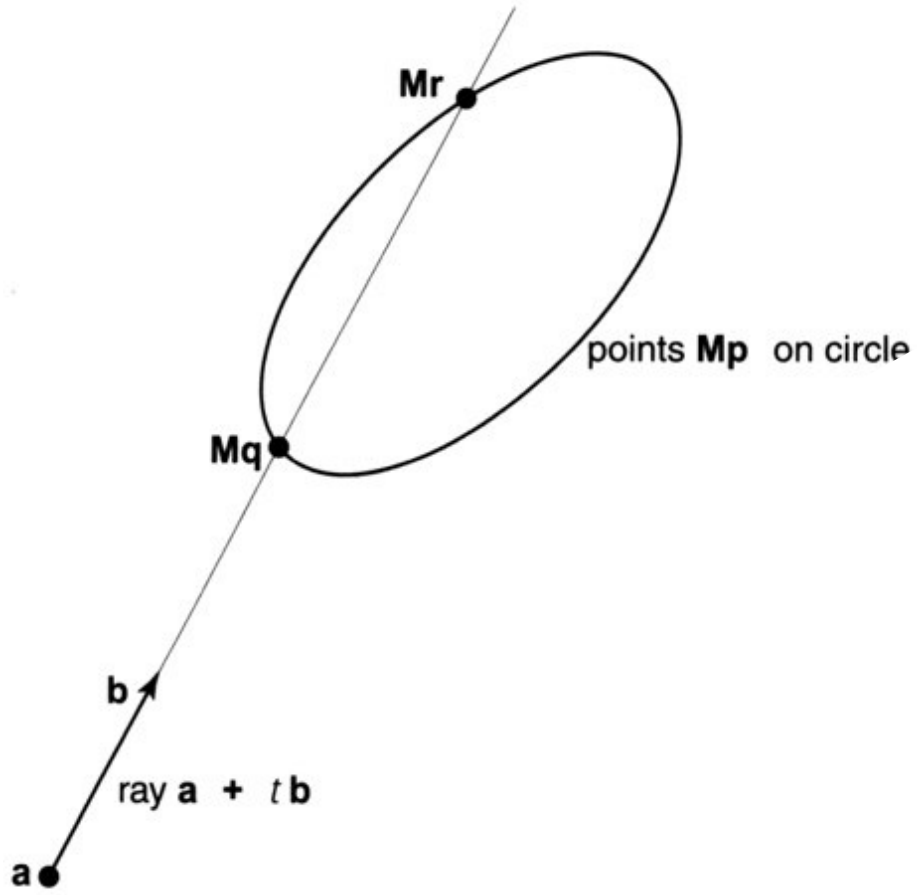
Topics

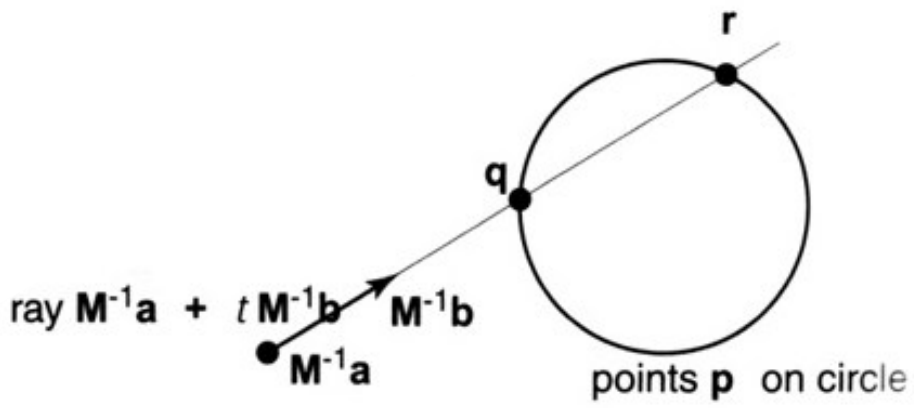
- Transformations in ray tracing
  - Transforming objects
  - Transformation hierarchies
- Ray tracing acceleration structures
  - Bounding volumes
  - Bounding volume hierarchies
  - Uniform spatial subdivision
  - Adaptive spatial subdivision

Transforming objects

- In modeling, we've seen the usefulness of transformations
  - How to do the same in RT?
- Take spheres as an example: want to support transformed spheres
  - Need a new Surface subclass
- Option 1: transform sphere into world coordinates
  - Write code to intersect arbitrary ellipsoids
- Option 2: transform ray into sphere's coordinates
  - Then just use existing sphere intersection routine

Intersecting transformed objects

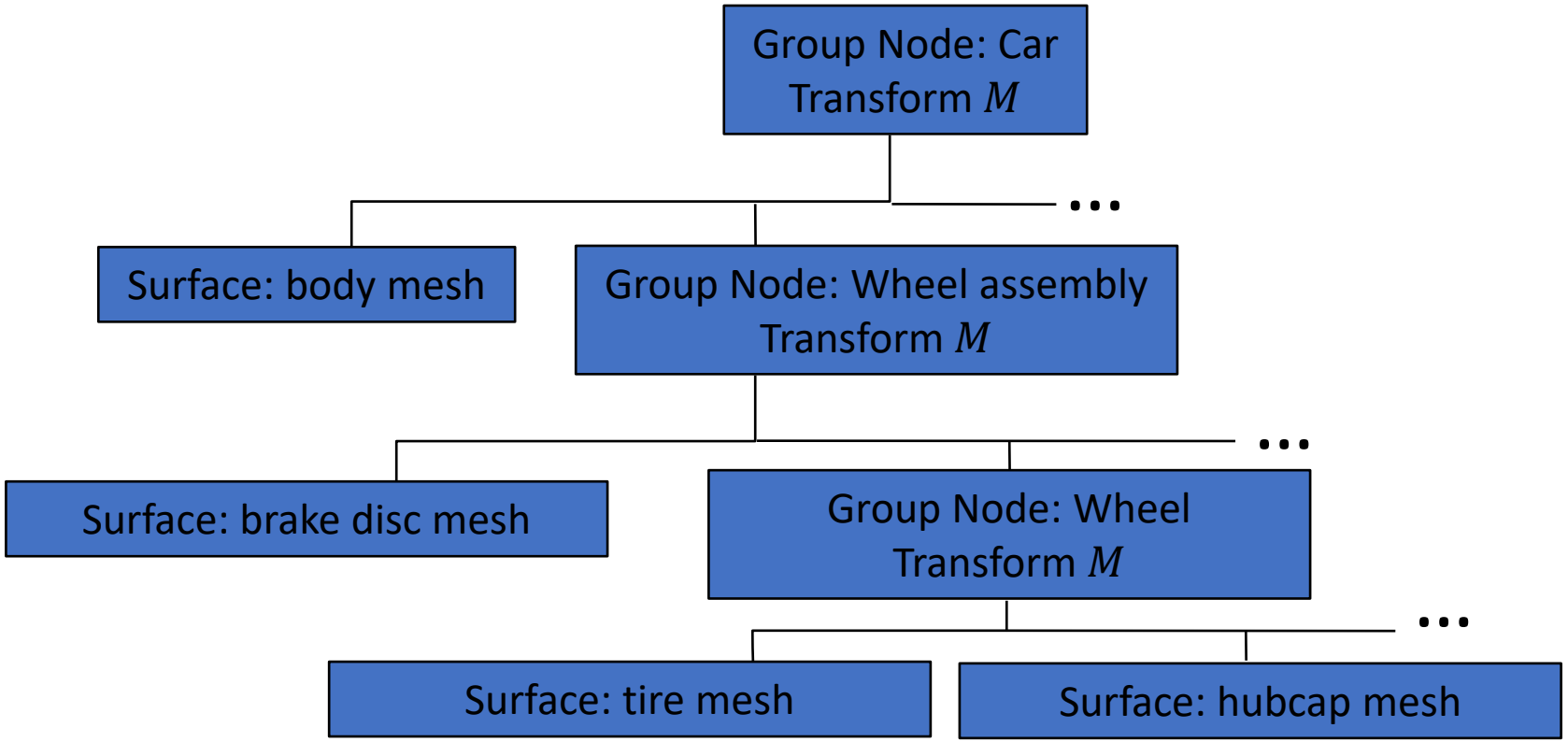




Implementing Transforms in Ray Tracing

- Create wrapper object for transforming surface
  - Has a transform M and a reference to a surface S, or set of surfaces
  - In A4, this is the `Node` object, and it groups several children (possibly nodes)
- To intersect with a transformed surface:
  - Transform ray to local coordinates by M^{-1}
  - For each surface, call intersect method, keep closest hit
  - Transform hit data back to global coords by M
    - Intersection point
    - Surface normal
    - Any other relevant data, e.g., material (maybe none)

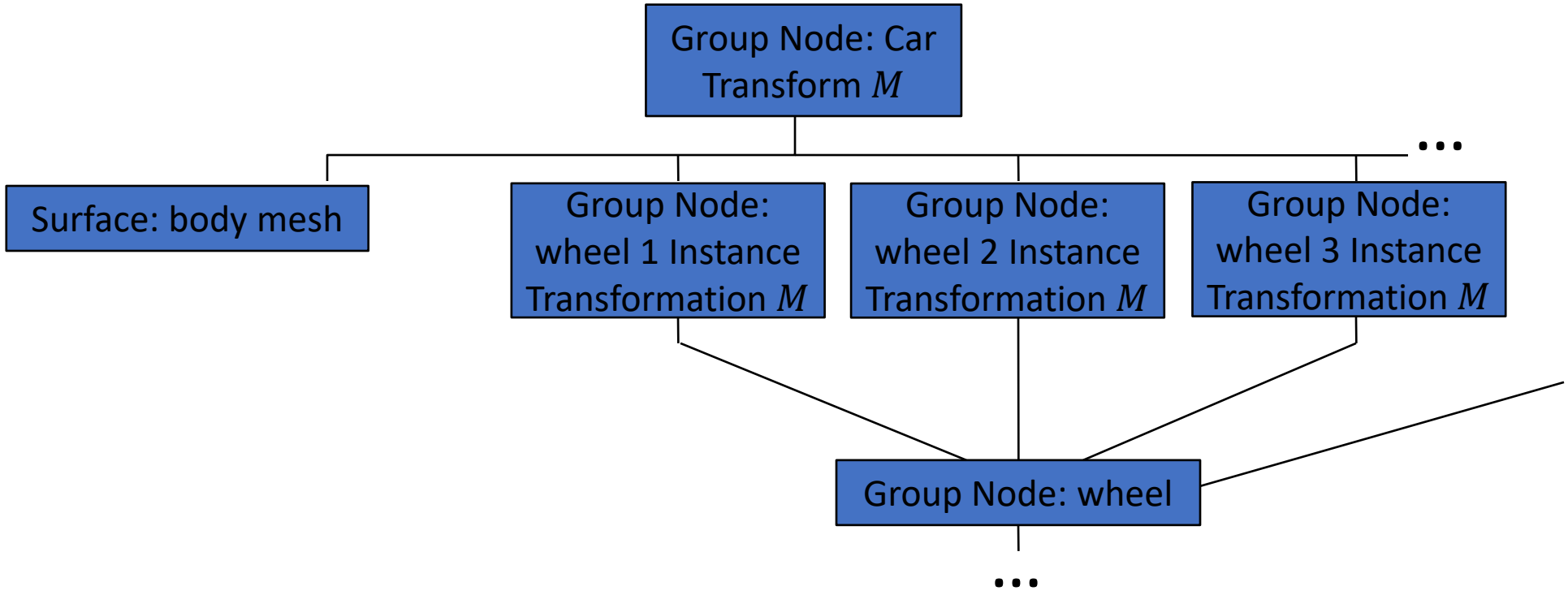
A transformation hierarchy



Instancing

- Anything worth doing is worth doing *n* times
- If we can transform objects, why not transform them several ways?
  - Many models have repeated subassemblies
    - Mechanical parts (wheels of car)
    - Multiple objects (chairs in classroom, …)
  - Nothing stops you from creating two transformation nodes that reference the same Surface
    - Allowing this makes the transformation tree into a DAG
      - (directed acyclic graph)
    - Mostly this is transparent to the renderer

Hierarchy with instancing



Hierarchies and performance

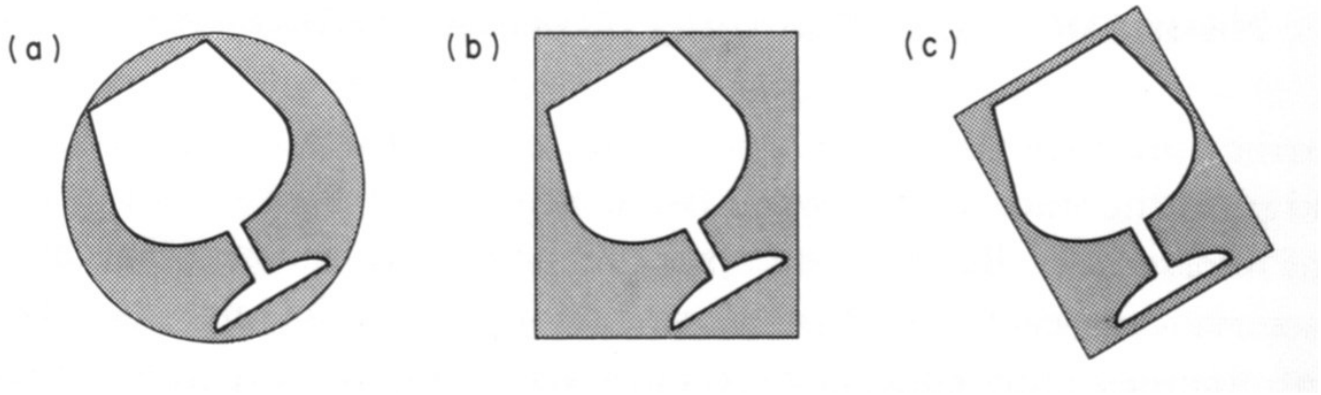
- Transforming rays is expensive
  - Minimizing tree depth will reduce ray transforms throughout hierarchy
  - Optimization: flatten on input
    - Push all transformations toward leaves
      (i.e., reparent leaves off root with appropriate transform
    - Triangle meshes can be transformed (i.e., transform vertices)
    - Many other surfaces can be transformed too (planes, spheres/quadrics, Bezier patches)
  - Internal group nodes are still required for instancing
    - Can’t push two transforms down to same child!
    - Could consider deep copy of subtrees, but if meshes are at leaves there may need to be many copies of transformed mesh vertices (could be better left as instances)

Ray tracing acceleration

- Ray tracing is slow. This is bad!
  - Ray tracers spend most of their time in ray-surface intersection methods
- Ways to improve speed
  - Make intersection methods more efficient
    - Yes, good idea. But only gets you so far
  - Call intersection methods fewer times
    - Intersecting every ray with every object is wasteful
    - Basic strategy:
      Efficiently find big chunks of geometry that definitely do not intersect a ray

Bounding volumes

- Quick way to avoid intersections: bound object with a simple volume
  - Object is fully contained in the volume
  - If it doesn't hit the volume, it doesn't hit the object
  - So test bounding volume first, then test object if it hits



[Glassner 89, Fig 4.5]

Bounding volumes

- Cost:
  - slightly more for hits and near misses,
  - ***much less*** for far misses
- Worth doing? It depends:
  - Cost of bvol intersection test should be small
    - Therefore use simple shapes (spheres, boxes, …)
  - Cost of object intersect test should be large
    - Bvols most useful for complex objects, e.g., large meshes, or large subtrees
  - Tightness of fit should be good
    - Loose fit leads to extra object intersections
    - Tradeoff between tightness and bvol intersection cost

Implementing bounding volume

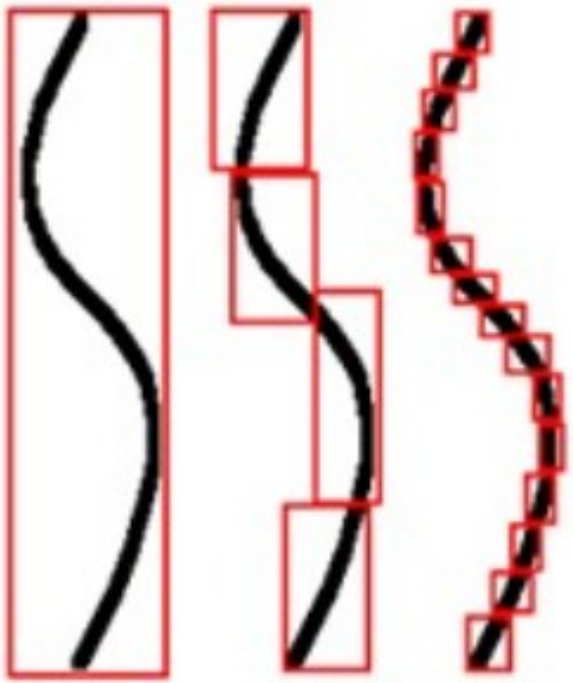
- Just add new Surface subclass, “BoundedSurface”
  - Contains a bounding volume and a reference to a surface
  - Intersection method:
    - Intersect with bvol, return false for miss
    - Return surface.intersect(ray)
  - Like transformation group nodes, might make sense to include transformation
  - This change is transparent to the renderer (only it might run faster)
- Note that all Surfaces must be able to supply bounding volumes for themselves

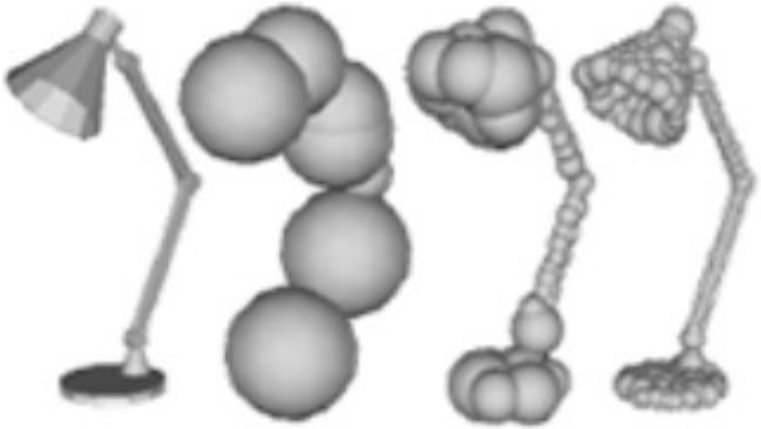
If it’s worth doing, it’s worth doing hierarchically!

- Bvols around objects may help
- Bvols around groups of objects will help
- Bvols around parts of complex objects will help
- Leads to the idea of using bounding volumes all the way from the whole scene down to groups of a few objects
- Meshes are a special case
  - We don’t add individual triangles into the DAG
  - We probably want to do something hierarchically at the level of an individual triangle soup!

Implementing a bvol hierarchy

- A BoundedSurface can contain a list of Surfaces
- Some of those Surfaces might be more BoundedSurfaces
  - Axis Aligned Bounding Boxes (AABB), Easy!
  - Spheres, can computed in expected O(n) time
    - WELZL, E. 1991. Smallest enclosing disks (balls and ellipsoids)
    - See python **miniball** module
  - Oriented Bounding Boxes (OBBs), and other types also possible
  - Bounding volumes and hierarchy is precomputation!
    - Effort to build hierarchy offset when casting large amounts of rays, or increased number of samples per pixel





Choice of bounding volumes

- Spheres
  - Easy and cheap to intersect,
  - Can compute tight bounds using Welzl's smallest enclosing disk algorithm
  - Not always so tight a wrap on geometry (e.g., consider a thin cylinder)
- Axis-aligned bounding boxes (AABBs)
  - Easy to and almost as cheap as spheres to intersect
  - Often tighter than spheres (esp. for axis-aligned models)
- Oriented bounding boxes (OBBs)
  - easy to intersect (but cost of transformation), tighter for arbitrary objects

Computing the bounding volumes

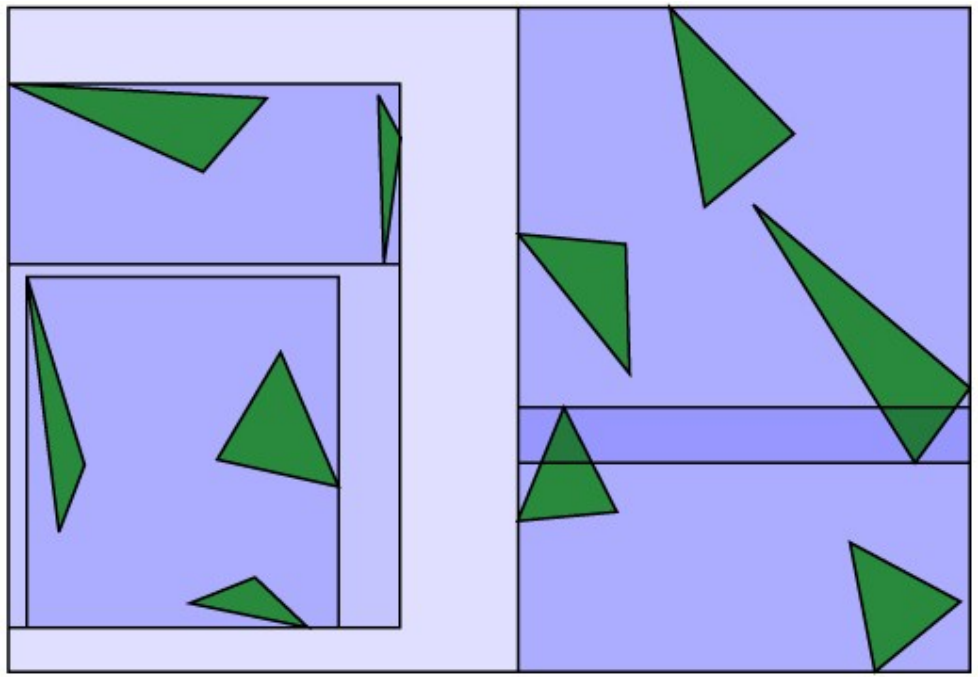
- For primitives
  - Sphere, cube? Probably easy for different bounding volumes
  - Meshes? Also easy for different bounding volumes
- For groups
  - Might not be so easy for certain bounding volumes (e.g., OBBs)
- For transformed surfaces
  - Does it make sense to bound a transformed sphere?
  - Otherwise, bounding a set of transformed spheres might not be so easy.

- ***Conservative bounds are probably OK (i.e., don't need to be tight) and can still help speed up rendering***

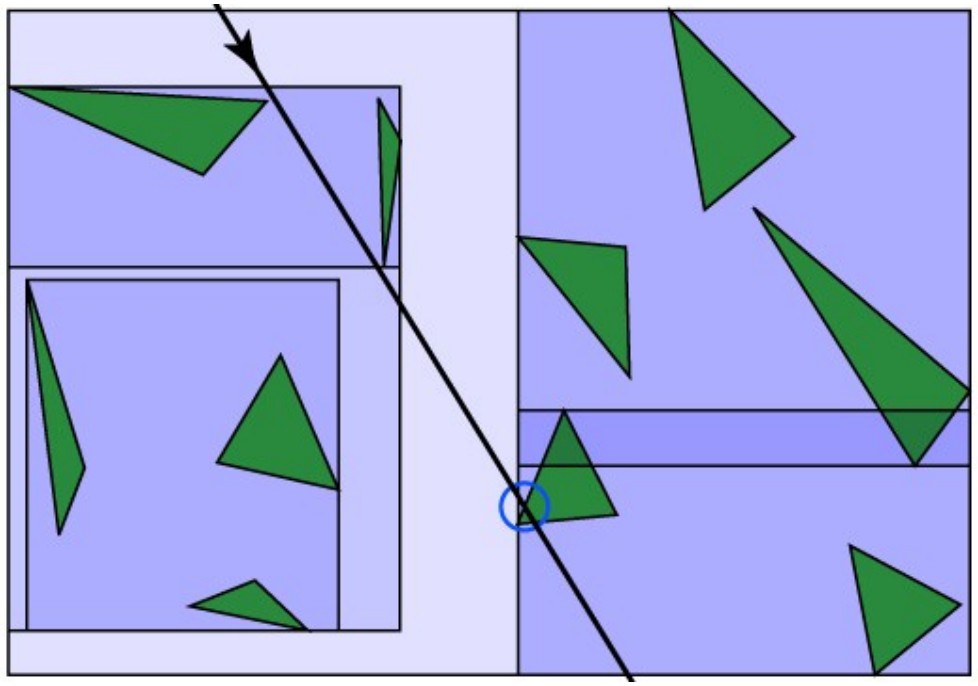
Hierarchy construction for triangle soup

- Consider a top down approach and a binary tree
  - Could alternatively use clustering and bottom up approaches
- Wrap entire set of primitives (i.e., all vertices) in bvol
- Choose a separating plane to divide the primitives into groups
  - Could be xy, yz, xz plane, could select plane normal based on direction with greatest spread of positions
  - Could divide at median, e.g., for a well balanced tree?
  - Could divide at midpoint, e.g., consider if this would improve culling of ray intersection tests and increase render speed
  - Repeat

BVH construction example, AABB tree



BVH ray-tracing example



Axis aligned bounding boxes

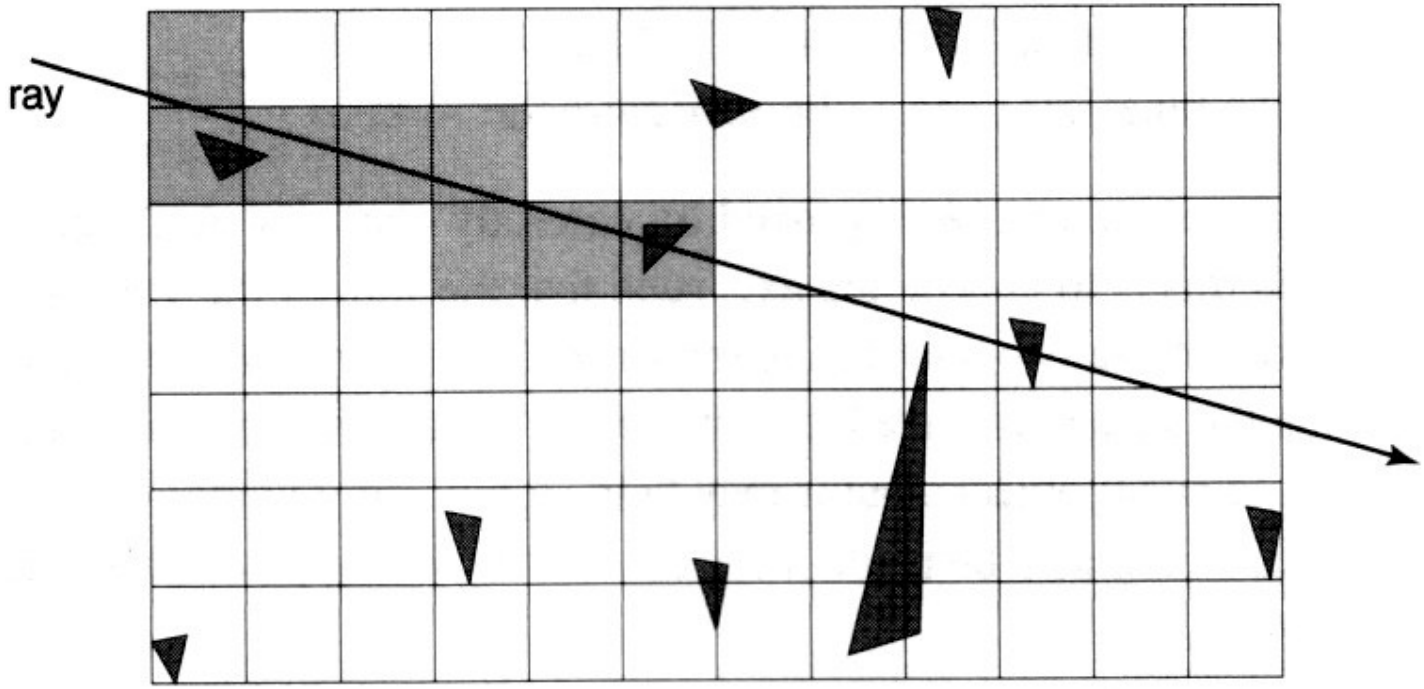
- Probably easiest to implement
- Computing for primitives
  - Cube: very straightforward!
  - Sphere, cylinder, etc.: easy!
  - Groups or meshes: min/max of component parts
- AABBs for transformed surface
  - Easy to do conservatively: bbox of the 8 corners of the bbox of the untransformed surface
- Can compute intersection as we saw previously as the intersection of 3 slabs (but don’t need normal)

Other acceleration techniques

- Consider regular grids, spatial hashing, k-d trees
- Let’s see a few examples

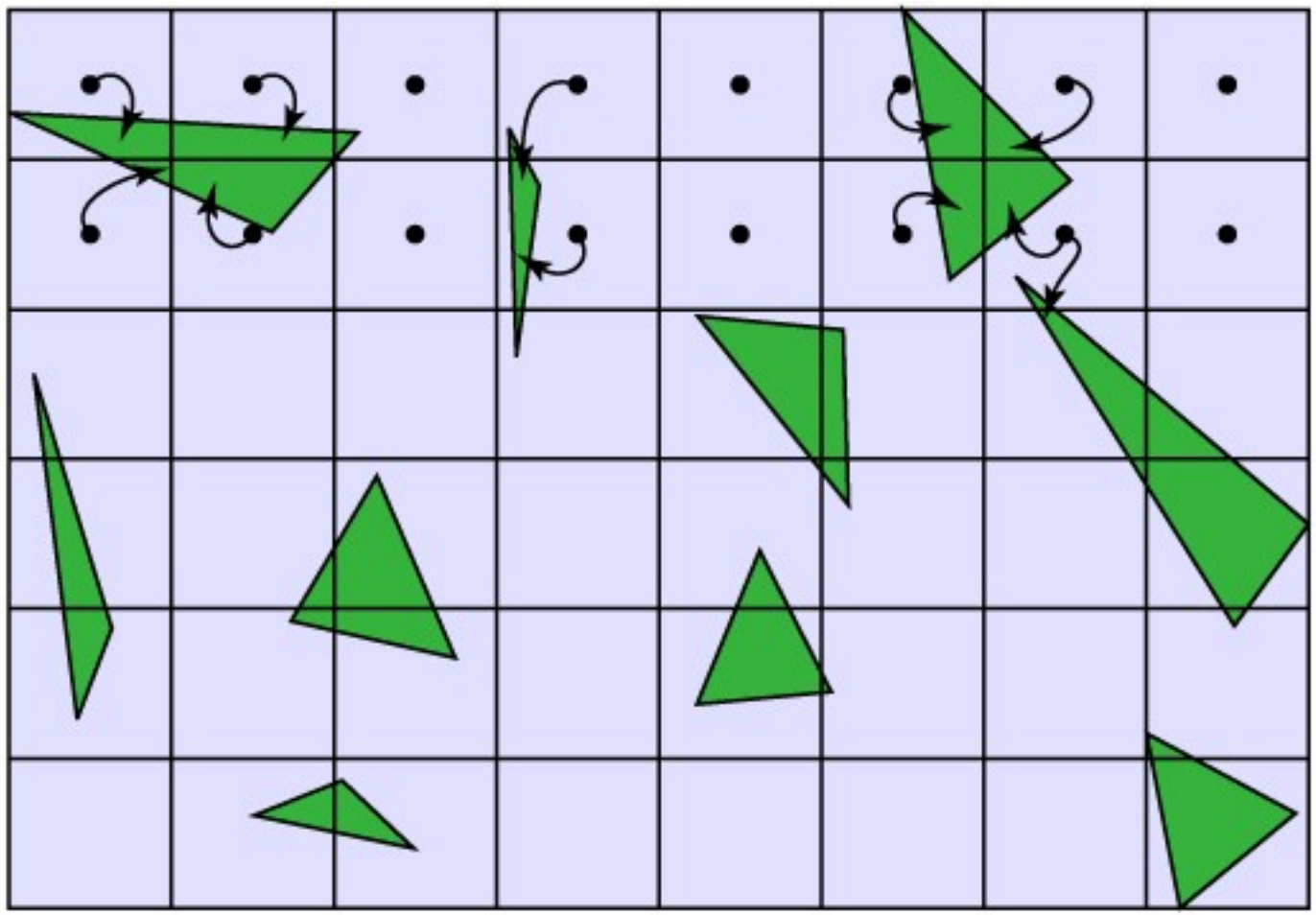
Regular space subdivision

- An entirely different approach: uniform grid of cells

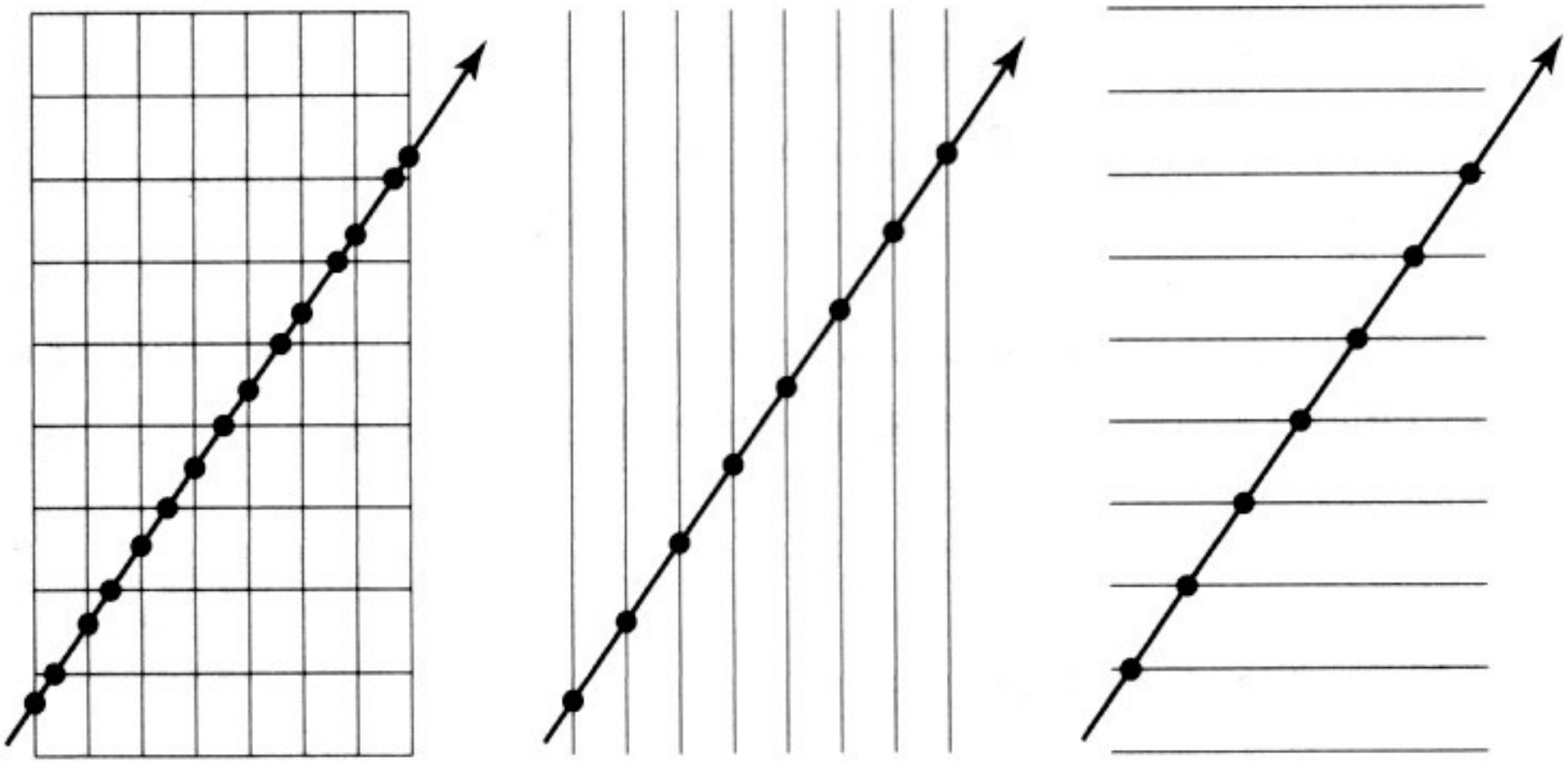


Regular grid example

- Grid divides space, not objects
- Use spatial hashing to avoid allocating a large volumetric grid

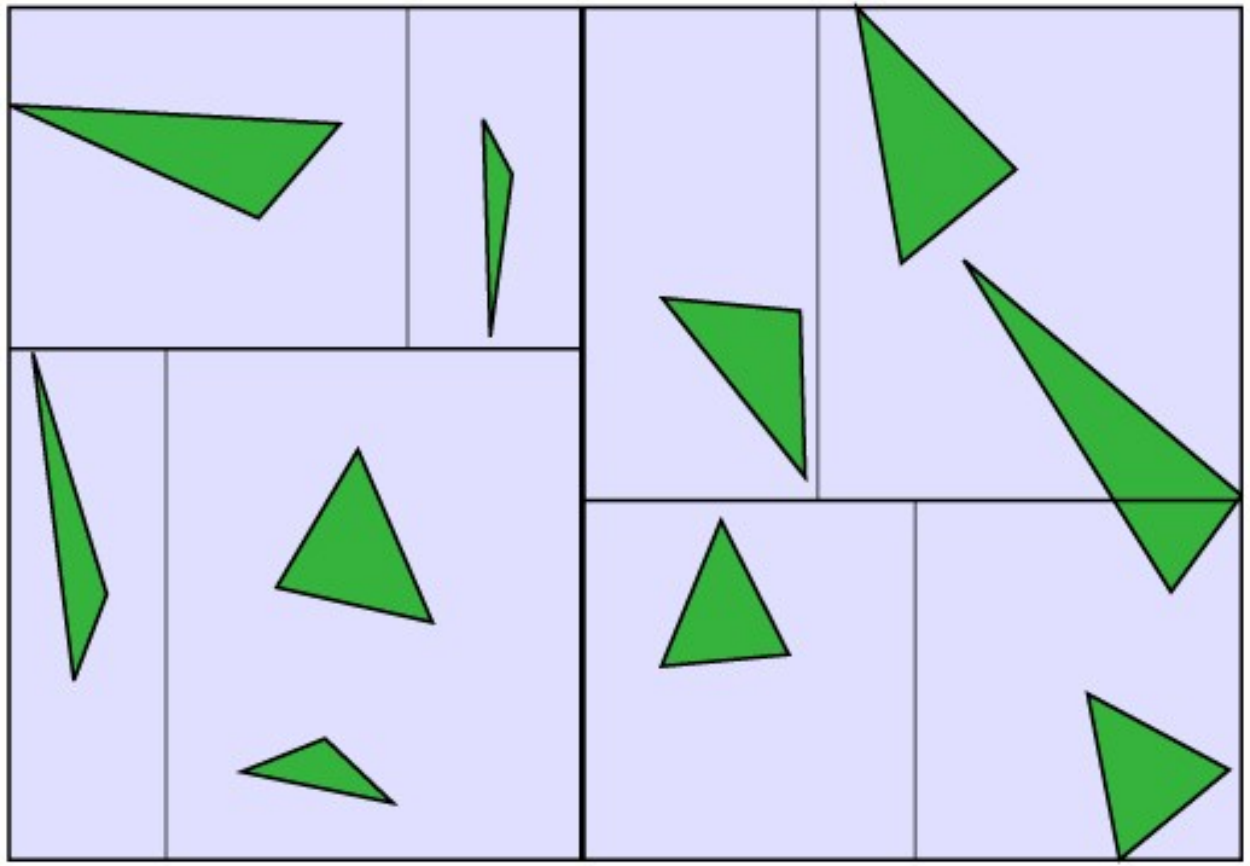


Traversing a regular grid



Non-regular space subdivision

- *k*-d Tree
  - subdivides space, like grid
  - adaptive, like BVH



Implementing acceleration structures

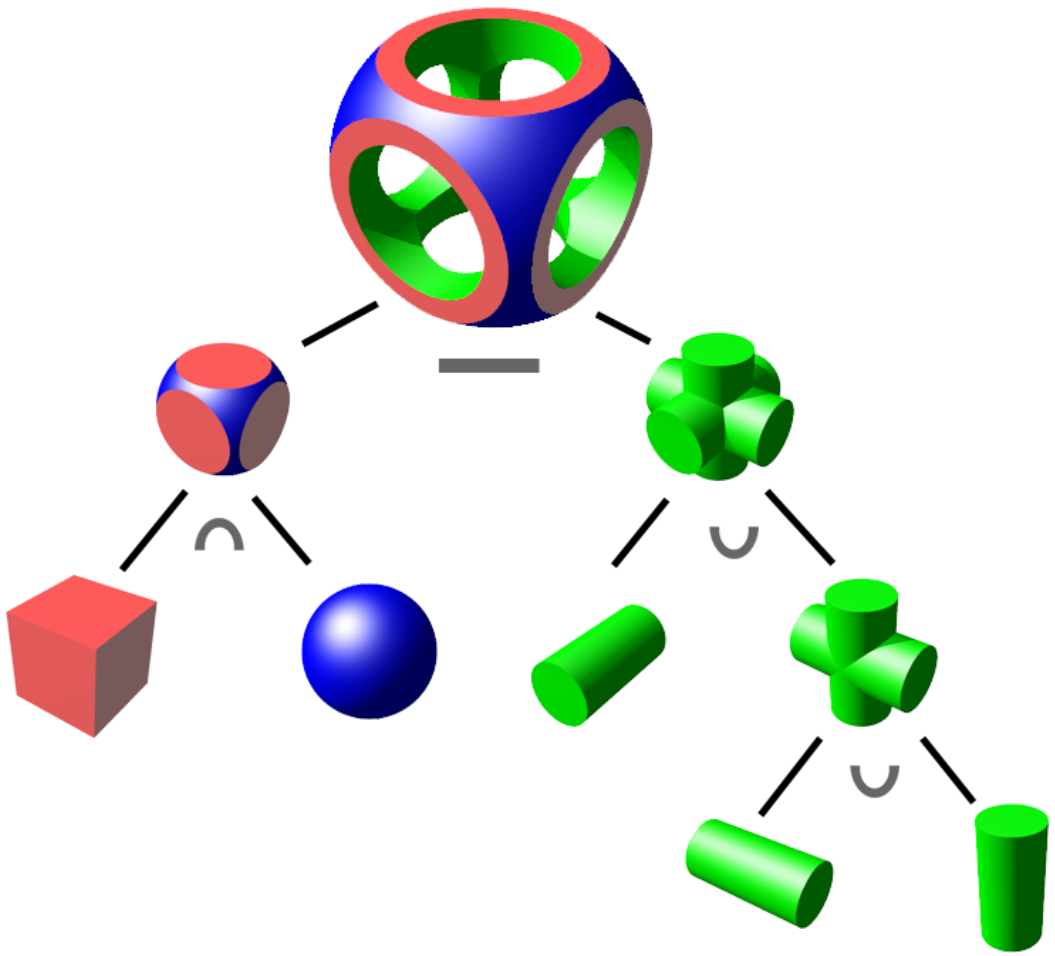
- Conceptually simple to build acceleration structure into scene structure
- Better engineering decision to separate them

Other ray tracing features

- Fresnel reflection and refraction
- Area light sources / soft shadows
- Super sampling for anti aliasing & jittering
- Depth of field blur / real lens modelling
- Motion blur
- Transparency and compositing
- Cartoon shaders
- Texture maps (colour, displacement, normal)
- Other implicit surfaces (metaballs)
- Subdivision surfaces / parametric surfaces
- Constructive solid geometry
- Perlin Noise

Constructive Solid Geometry

- Exactly like intersecting a cube using 3 slabs
- Assemble binary set operations in a tree (works nicely with a dag scene graph)
- Ray intersection class ***must maintain a list of all intersections*** instead of closest intersection.



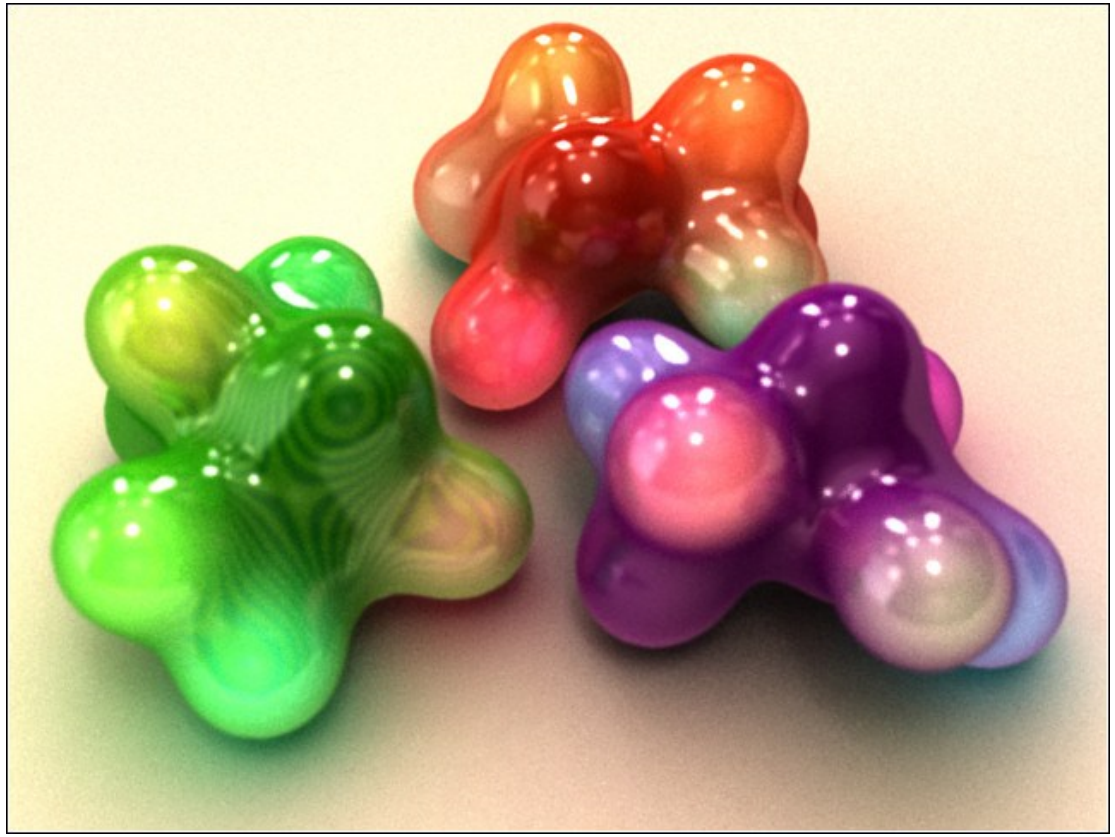
Meta Balls

- Implicit function (other (better) choices exist!)

$$\sum_{i=0}^{n} f_i(\boldsymbol{x}) = \text{threshold}$$

$$f_i(\boldsymbol{x}) = \frac{1}{\|\boldsymbol{x} - \boldsymbol{x}_i\|^2}$$

- ***Consider the picture at right***
  - ***What will be the normal?***
  - ***How do you find the intersection?***
  - ***Where do the colours come from?***



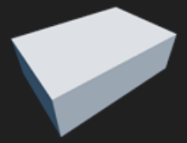
Alternatively, 3D distance functions

https://iquilezles.org/articles/distfunctions/

- Many of the shapes you might like to design can be created with implicit surfaces using signed distance functions, or signed field functions

Sphere - exact (https://www.shadertoy.com/view/Xds3zN)

```
float sdSphere( vec3 p, float s )
{
  return length(p)-s;
}
```



Box - exact (Youtube Tutorial with derivation: https://www.youtube.com/watch?v=62-pRVZuS5c)

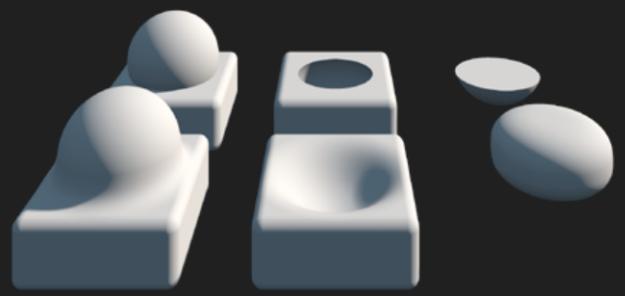
```
float sdBox( vec3 p, vec3 b )
{
  vec3 q = abs(p) - b;
  return length(max(q,0.0)) + min(max(q.x,max(q.y,q.z)),0.0);
}
```

Can blend distance functions like meta balls

https://iquilezles.org/articles/distfunctions/

Smooth Union, Subtraction and Intersection - bound, bound, bound

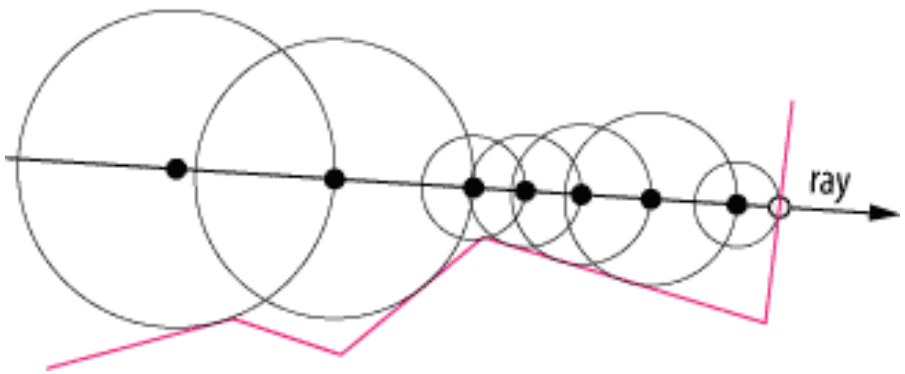
Blending primitives is a really powerful tool - it allows to construct complex and organic shapes without the geometrical semas that normal boolean operations produce. There are many flavors of such operations, but the basic ones try to replace the min() and max() functions used in the opUnion, opSubstraction and opIntersection above with smooth versions. They all accept an extra parameter called **k** that defines the size of the smooth transition between the two primitives. It is given in actual distance units. You can find more details in the smooth minimum article article in this same site. You can code here: https://www.shadertoy.com/view/lt3BW2

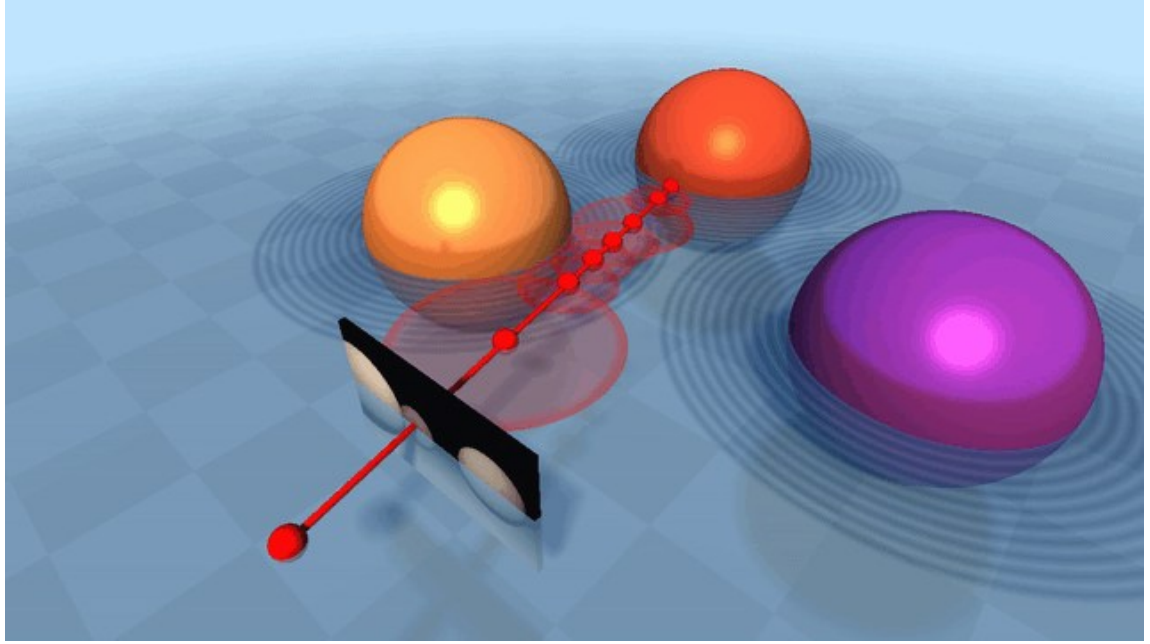


```
float opSmoothUnion( float d1, float d2, float k )
{
    float h = clamp( 0.5 + 0.5*(d2-d1)/k, 0.0, 1.0 );
    return mix( d2, d1, h ) - k*h*(1.0-h);
}

float opSmoothSubtraction( float d1, float d2, float k )
{
    float h = clamp( 0.5 - 0.5*(d2+d1)/k, 0.0, 1.0 );
    return mix( d2, -d1, h ) + k*h*(1.0-h);
}

float opSmoothIntersection( float d1, float d2, float k )
{
    float h = clamp( 0.5 - 0.5*(d2-d1)/k, 0.0, 1.0 );
    return mix( d2, d1, h ) + k*h*(1.0-h);
}
```

Ray marching to find intersection surface

- Walk along the ray looking for intersection with surface
- Given a **signed distance function**, then at any point we can compute the **distance** and know that we can step ***at least this far*** along the ray

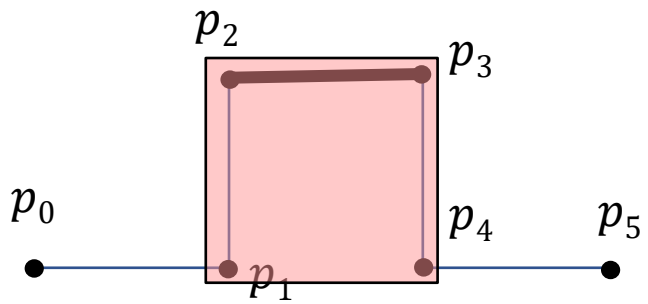


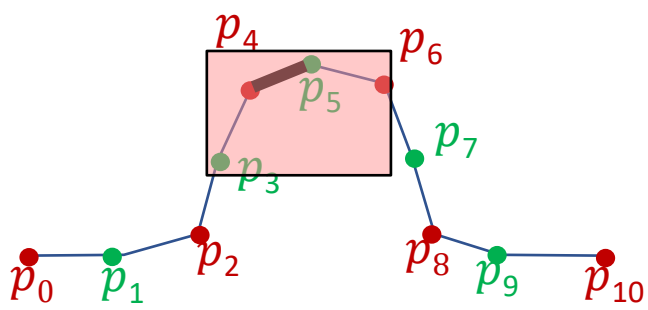


See also https://iquilezles.org/articles/raymarchingdf/

Subdivision Surfaces

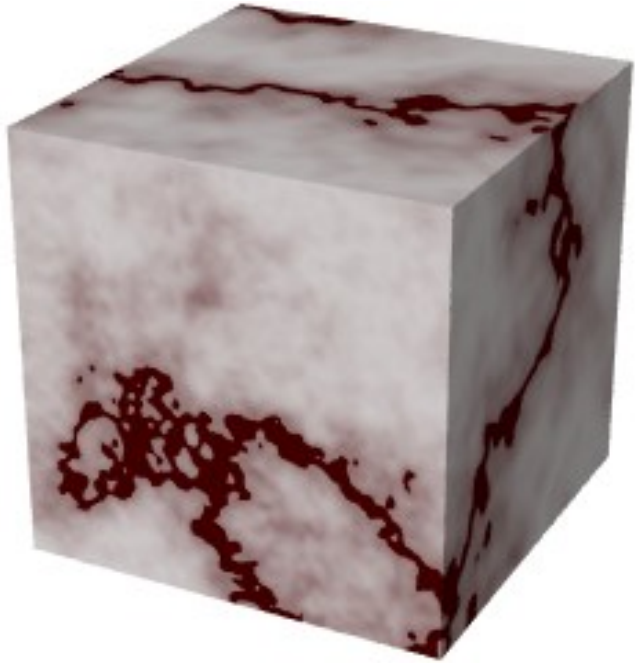
- Many subdivision schemes simply involve a ***convex combination of points***
  - Chaikin, Lane-Reisenfeld, Loop, Catmull-Clark
- Convex hull of points can be used for ray-intersection test
  - Recursively subdivide checking with bounding box of all child faces
  - Once sufficiently flat (or small), use ray polygon intersection

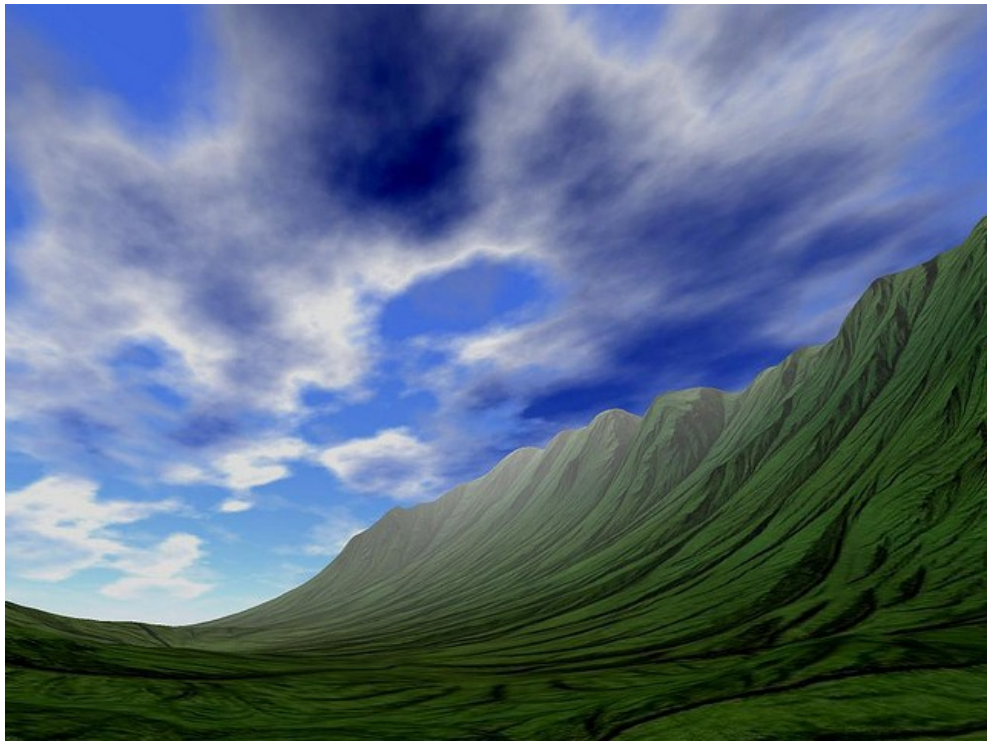




Perlin Noise

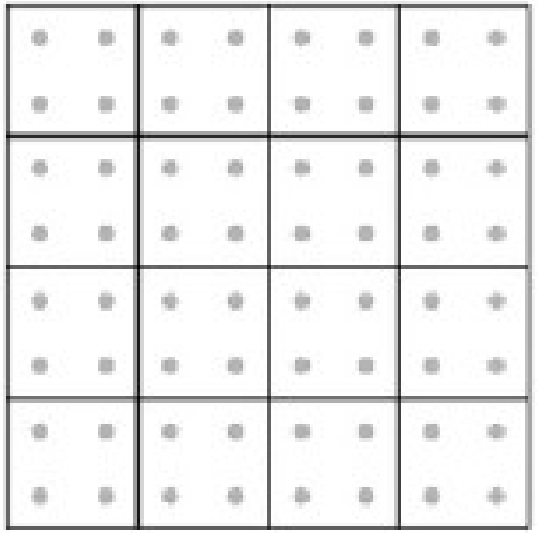
- Useful for clouds, marble, and other random varying effects
- Code is very small… noise is easy to compute!

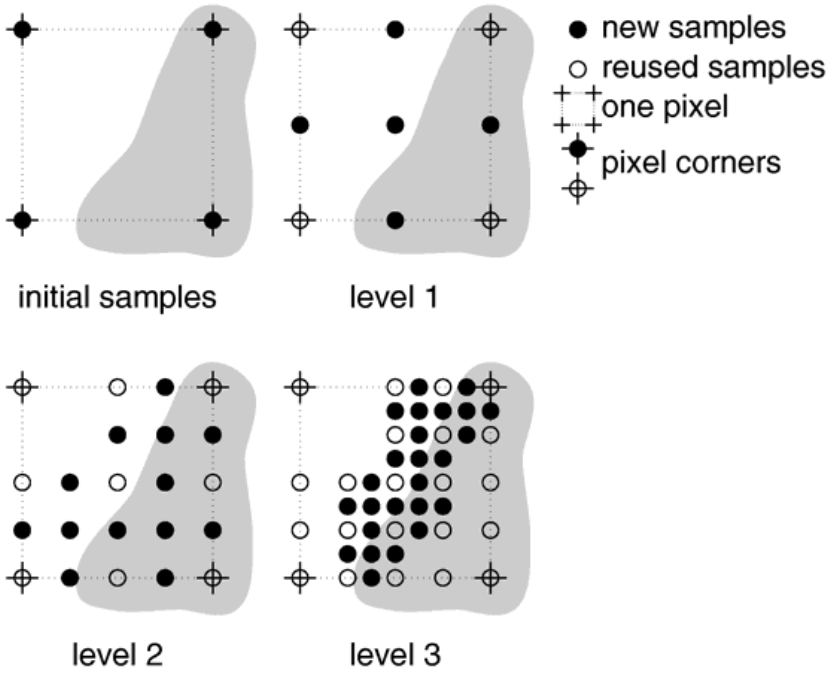




Super Sampling and Jittering Samples

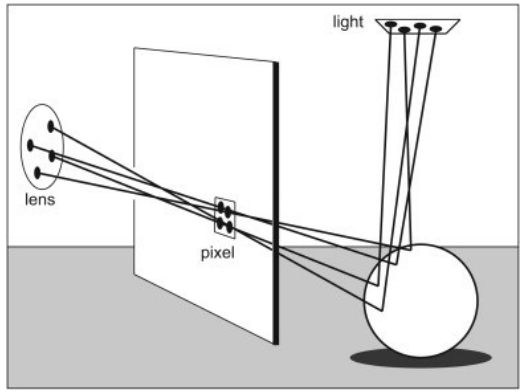
- Cast more rays through sub pixel positions.
- Watch out for aliasing!
  - Jittering samples will replace aliasing with noise, which is preferable
  - Noise added to sub-pixel position should not be too large!
- Super sampling can be done adaptively based on scene complexity at the pixel

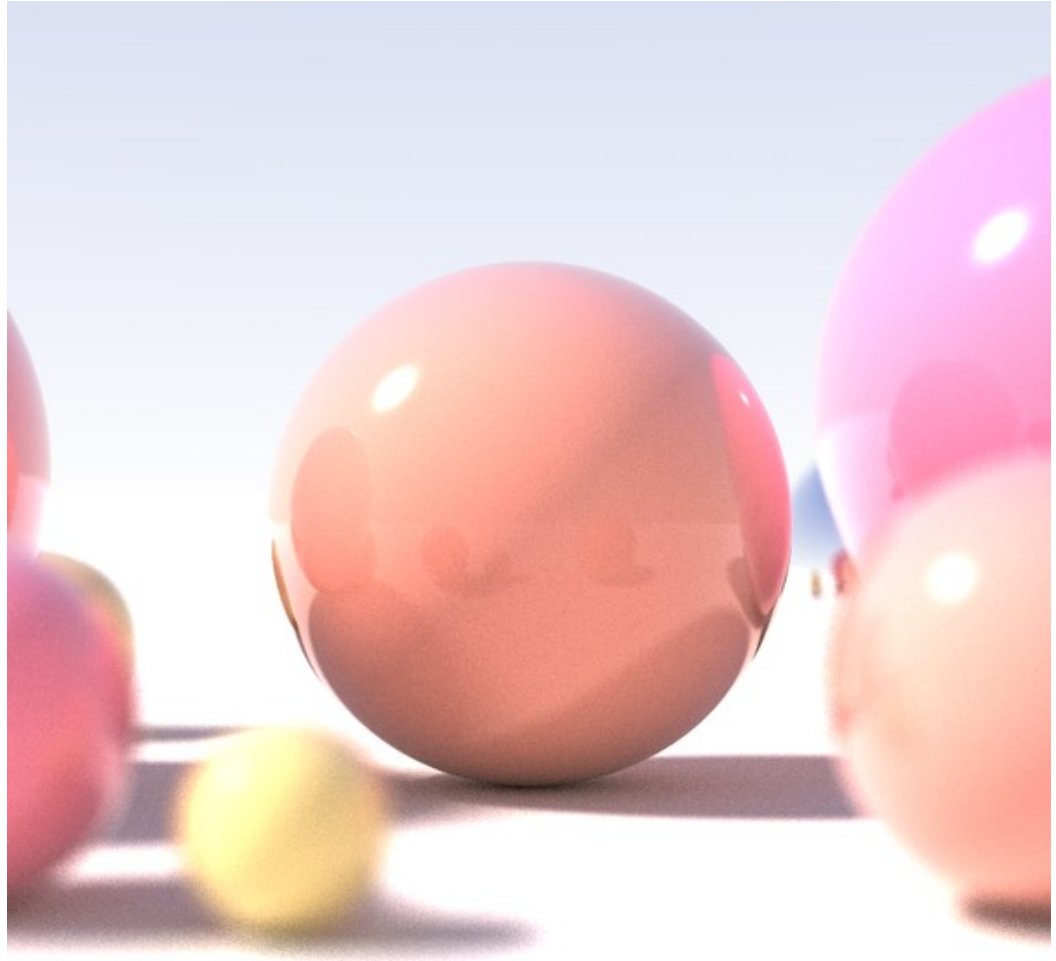




Depth of Field, Area Lights

- Cast more rays!
  - Sample different ***random positions*** on lens aperture by changing the eye position
  - For an area light source, do shadow ray test and lighting computation for a different ***random positions*** on the area light source.





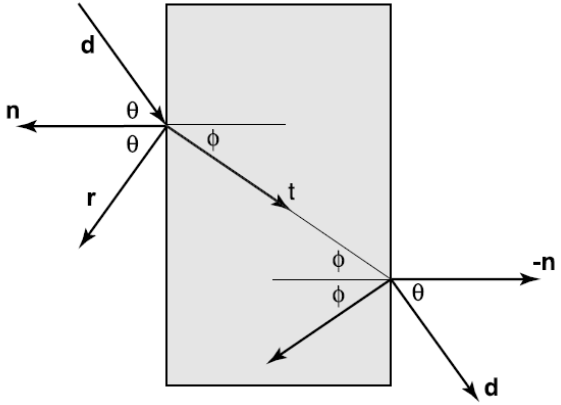
Motion Blur

- Knowing object trajectories over a time interval, cast rays at different times and average
- Can also just average a sequence of images, but this would produce temporal aliasing!
- Can also be smart about not casting few rays where nothing is in motion

[Thomas Porter 1984]

7
3
4

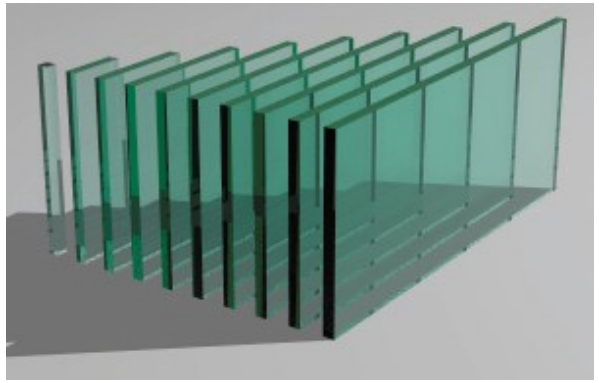
Reflection Refraction Absoption (FCG 13.1)



- Snell's law: relationship between incident and transmitted light angles from refractive indices

$$n \sin\theta = n_t \sin\phi$$

  - Solution can involve total internal reflection



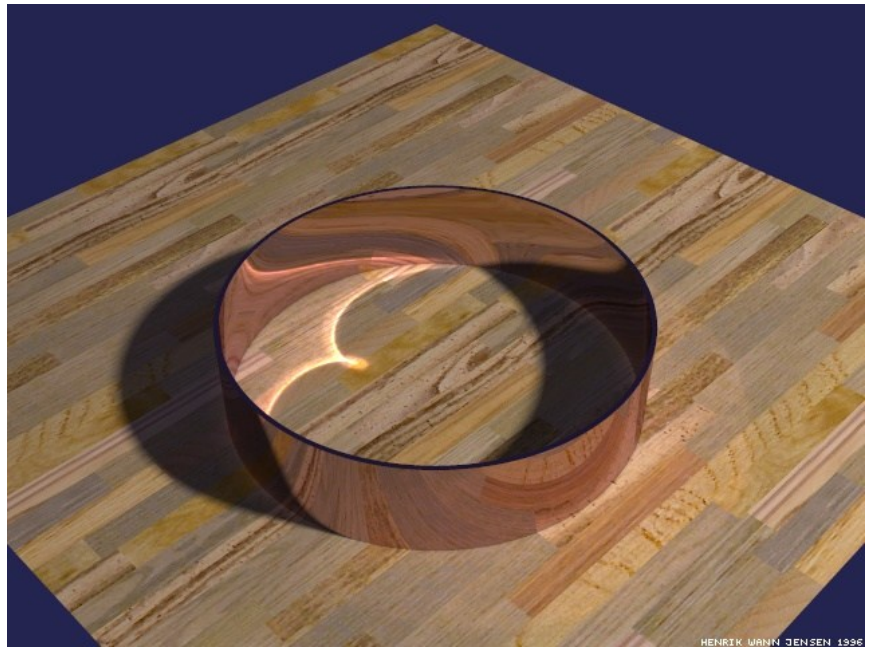
- Fresnel equations: how much light should be reflected dependent on the incident angle
  - Schlick approximation commonly used instead

- Beer's law: light attenuation by a according to distance d traveled in a tinted medium

$$I(d) = I(0) \exp - \ln(a)\, d$$

Caustics

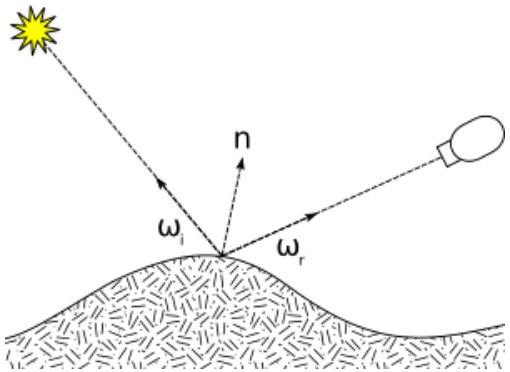
- Reflection and refraction scatters light too
  - Very challenging without photon mapping, or path tracing, etc.



Anisotropic BRDFs

- Can use a varying reflectance function for computing the amount of reflected light

- BRDF is a bidirectional reflectance distribution function $f_r(\omega_i, \omega_r)$

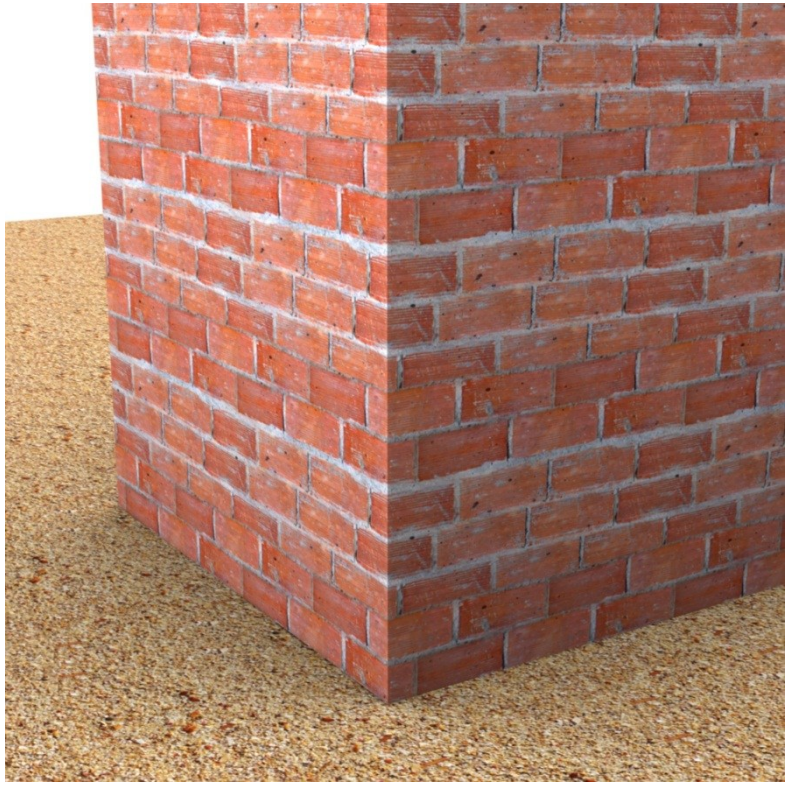


Cartoon Shaders

- Change lighting computations to produce a smaller number of colours
- Add black edges at boundaries between objects

Texture Maps

- Can download texture images from a variety of repositories.
- Easy to texture certain primitives (cubes, spheres, planes)
- Meshes often have texture coordinates
- ***Sampling is an issue!!***
  - Resolution may be too high or too low giving a poor quality result
  - Solve with filtering, or more samples, or adaptive sampling



Review and More Information

- FCG Chapters 12 and 13
- More information:
  - https://iquilezles.org/articles/
    - 3D SDFs
    - Raymarching SDFs
    - And lots more to explore